TI-2400 Processamento de Linguagem Natural

ANTONIO CARVALHO - TREINAMENTOS

Numpy

O que é Numpy

NumPy é uma biblioteca em Python que provê matrizes multidimensionais, assim como vários objetos derivados, e diversas rotinas para operações rápidas em matrizes.

Esta biblioteca é fundamental para computação científica com Python

NumPy documentation — NumPy v1.25 Manual

Diferenças entre o Numpy e as listas sequenciais em Python

- As matrizes do Numpy possuem um tamanho fixo já no momento de criação. Qualquer alteração no tamanho de uma matriz, ira ocasionar na criação de uma nova matriz e no descarte da original.
- Os elementos da matriz precisam ser todos do mesmo tipo de dado
- As matrizes facilitam operações matemáticas em grandes volumes de dados

What is NumPy? — NumPy v1.25 Manual

Arrays, Propriedades básicas

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]], dtype=np.float32)
print(a.ndim, a.shape, a.dtype)
```

1. Arrays podem conter qualquer número de dimensões incluindo zero (um scalar).
2. Arrays são tipadas: np.uint8, np.int64, np.float32, np.float64
3. Arrays são densas. Todos os elementos do array existem e são do mesmo tipo.

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> np.ones((3,5),dtype=np.float32)
array([[ 1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.]], dtype=float32)
```

```
>>> np.zeros((6,2),dtype=np.int8)
array([[0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0]], dtype=int8)
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> np.arange(1334,1338)
array([1334, 1335, 1336, 1337])
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> A = np.ones((2,3))
>>> B = np.zeros((4,3))
>>> np.concatenate([A,B])
array([[ 1.,  1.,  1.],
       [ 1.,  1.,  1.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.]])
>>>
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> A = np.ones((4,1))
>>> B = np.zeros((4,2))
>>> np.concatenate([A,B], axis=1)
array([[ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.]])
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> A
array([[ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5]], dtype=float32)
>>> print(A.astype(np.uint16))
[[4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]]
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> a = np.ones((2,2,3))
>>> b = np.zeros_like(a)
>>> print(b.shape)
```

Arrays, criação

np.ones, np.zeros

np.arange

np.concatenate

np.astype

np.zeros_like, np.ones_like

np.random.random

```
>>> np.random.random((10,3))
array([[ 0.61481644,  0.55453657,  0.04320502],
       [ 0.08973085,  0.25959573,  0.27566721],
       [ 0.84375899,  0.2949532 ,  0.29712833],
       [ 0.44564992,  0.37728361,  0.29471536],
       [ 0.71256698,  0.53193976,  0.63061914],
       [ 0.03738061,  0.96497761,  0.01481647],
       [ 0.09924332,  0.73128868,  0.22521644],
       [ 0.94249399,  0.72355378,  0.94034095],
       [ 0.35742243,  0.91085299,  0.15669063],
       [ 0.54259617,  0.85891392,  0.77224443]])
```

Arrays, criação

```
array = np.array([[0,1,2],[2,3,4]])
[[0 1 2]
 [2 3 4]]

array = np.zeros((2,3))
[[0. 0. 0.]
 [0. 0. 0.]]

array = np.ones((2,3))
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
array = np.eye(3)
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

array = np.arange(0, 10, 2)
[0, 2, 4, 6, 8]

array = np.random.randint(0, 10,
(3,3))
[[6 4 3]
 [1 5 6]
 [9 8 5]]
```

Arrays, problemas

Precisam ser densas, sem buracos

Precisam ser de um único tipo

Não é possível combinar arrays de tipos diferentes

```
>>> np.ones([7,8]) + np.ones([9,3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operands could not be broadcast together
 with shapes (7,8) (9,3)
```

Exercícios

1. Importe a biblioteca NumPy e crie um array NumPy de números inteiros de 1 a 10.
2. Crie um array Numpy de números float aleatórios entre 0 e 1 nas seguintes dimensões 300 x 200 x 3
3. Crie um array de duas dimensões para guardar notas de alunos de uma disciplina, sendo que existem 20 alunos na sala e cada aluno possuirá 3 notas na disciplina. O array deverá conter números aleatórios inteiros entre 1 e 10 (inclusive)

Formato

```
a = np.array([1,2,3,4,5,6])
a = a.reshape(3,2)
a = a.reshape(2,-1)
a = a.ravel()
```

1. Número total de elementos não pode ser modificado.
2. Utilize -1 para inferior o formato do eixo
3. Row-major por padrão

Valores retornados

Funções do Numpy retornam ambas **Views** e **Copias**

As **Views** compartilham os dados com o Array original, como referências em Java/C++. Ao alterar os ítens em uma **View** modificam os valores no Array original.

A documentação do numpy (https://numpy.org/doc/stable/reference/) mostra quais funções retornam **Views** ou **Cópias**

np.copy, **np.view** fazem de maneira explicita cópias ou views.

Transposição

```
a = np.arange(10).reshape(5,2)
a = a.T
a = a.transpose((1,0))
```

np.transpose permuta os eixos.

a.T transpõem os dois primeiros eixos.

Salvando e carregando arrays

```
np.savez('data.npz', a=a)
data = np.load('data.npz')
a = data['a']
```

1. Arquivos NPZ podem conter múltiplos Arrays

Exercícios

1. Crie um array NumPy com 12 elementos e o redimensione em uma matriz 3x4. Em seguida, realize a transposição dessa matriz.

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações locais modificam o Array

```
>>> a
array([1, 2, 3])
>>> b
array([ 4,  4, 10])
>>> a * b
array([ 4,  8, 30])
```

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações locais modificam o Array

```
>>> a
array([[ 0.93445601,  0.42984044,  0.12228461],
       [ 0.06239738,  0.76019703,  0.11123116],
       [ 0.14617578,  0.90159137,  0.89746818]])
>>> a > 0.5
array([[ True, False, False],
       [False,  True, False],
       [False,  True,  True]], dtype=bool)
```

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações locais modificam o Array

```
>>> a
array([[ 4, 15],
       [20, 75]])
>>> b
array([[ 2,  5],
       [ 5, 15]])
>>> a /= b
>>> a
array([[2, 3],
       [4, 5]])
```

Operadores matemáticos

O resultado das operações aritméticas pode retornar valores com precisão maior

uint64 + uint16 => uint64

float32 / int32 => float32

Exercícios

1. Crie dois arrays NumPy de mesma forma e realize as quatro operações aritméticas básicas (+, -, *, /) entre eles.

Matemática, funções universais

Também chamadas de **ufuncs**

Examples:

- np.exp
- np.sqrt
- np.sin
- np.cos
- np.isnan

```
>>> a
array([[ 1,  4],
       [ 9, 16],
       [25, 36]])
>>> np.sqrt(a)
array([[ 1.,  2.],
       [ 3.,  4.],
       [ 5.,  6.]])
```

Indexação

```
x[0,0]    # Elemento superior esquerdo
x[0,-1]   # primeira linha, última coluna
x[0,:]    # primeira linha (várias células)
x[:,0]    # primera coluna (várias células)
```

Nota:

- Indices Multi-dimensionais são separados por virgula (como uma tuple)

Indexação

```
I[1:-1,1:-1]        # selecione tudo, exceto a
borda de um pixel

I = I[:,:,::-1]     # troca ordem de canal

I[I<10] = 0         # definir valores menores do
que 10 para 0

I[[1,3], :]         # seleciona a 2ª e a 4ª linha
```

1. Fatias do array são **Views**. Modificar um slice, altera o array original.

Fatias do array

Sintaxe: inicio:fim:passo

```
a = np.array(list(range(10)))
a[:3] # indices 0, 1, 2
a[-3:] # indices 7, 8, 9
a[3:8:2] # indices 3, 5, 7
a[4:1:-1] # indices 4, 3, 2
```

Fatias do array

```
arr = np.arange(10)
print(arr)        # [0 1 2 3 4 5 6 7 8 9]

arr_slice = arr[5:8]
print(arr_slice)        # [5 6 7]
arr_slice[1] = 12345
print(arr)              # [    0     1     2     3     4     5 12345     7     8     9]
arr_slice[:] = 64
print(arr)              # [ 0  1  2  3  4 64 64 64  8  9]
```

Eixos

```
a.sum() # soma todas as células
a.sum(axis=0) # soma sobre as linhas
a.sum(axis=1) # soma sobre as colunas
a.sum(axis=1, keepdims=True)
```

1. Utilize o parâmetro **axis** para controlar qual eixo o Numpy irá operar
2. Normalmente o eixo especificado vai desaparecer, usar o **keepdims** mantem todas as dimensões

Dicas para evitar problemas

1. Saiba quais são seus tipos de dados.
2. Verifique se você tem uma visualização ou uma cópia.
3. Use matplotlib para verificações de integridade.
4. Use o debug para verificar cada etapa do seu cálculo.
5. Conheça a diferença entre **np.dot** e **np.mult**

Exercícios

1. Crie um array NumPy de números de 1 a 20 e, em seguida, imprima os elementos pares utilizando indexação e fatiamento.
2. Crie um array NumPy de números aleatórios entre 1 e 100. Em seguida, selecione e exiba apenas os números maiores que 50.
3. Crie um array NumPy com 10 números aleatórios e calcule a média, mediana e desvio padrão desses números.